Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Introducción a la Programación
CE-1101

Proyecto 1 – Imaginary Battle
Documentación externa

Joshua Sharbell Andrade Pérez
2026102162

Profesor Santiago Ramírez y Profesor Ellioth Ramírez.

I Semestre 2026.

2 de mayo de 2026

Introducción

El proyecto Imaginary Battle consiste en el desarrollo de un videojuego de peleas por turnos programado en Python. Los objetivos técnicos principales se centran en la implementación de funciones recursivas para la lógica de combate y el uso de la librería Tkinter para el diseño de una Interfaz Gráfica de Usuario (IGU) funcional.

Además del uso y desarrollo de estas herramientas, el proyecto busca fortalecer las destrezas de redacción técnica. Esto implica la creación de una documentación profesional que sea capaz de explicar con claridad la arquitectura del software, garantizando que el código no solo sea funcional, sino también comprensible y bien estructurado bajo estándares de ingeniería.

Arquitectura del Sistema

El proyecto se estructuró bajo una organización modular para separar estrictamente la lógica del juego del diseño visual. Esta arquitectura diferencia el motor lógico, ubicado en el paquete /Logic, de la interfaz gráfica de usuario desarrollada en Tkinter. El sistema se centraliza en el archivo main.py como punto de entrada único en la raíz del proyecto, garantizando un orden jerárquico que facilita la escalabilidad y el mantenimiento. Al aislar los procesos, se logra un bajo acoplamiento, permitiendo que las funciones recursivas operen de forma independiente a los eventos de la interfaz. Esto asegura que cualquier modificación técnica en las reglas de combate o en la base de datos de personajes no afecte la integridad del flujo visual del juego, así facilitando para el desarrollador la modificación de mecánicas.

Aunque el proyecto está organizado en módulos separados para la lógica (/Logic) y la interfaz (/Interfaz), ambos componentes se comunican a través de una estructura compartida llamada estado_juego. Esta variable global almacena información importante como el progreso, los personajes seleccionados y el puntaje, permitiendo que la interfaz refleje en tiempo real los cambios realizados por el motor lógico.

Implementación de la Recursividad

La implementación de la recursividad en este proyecto viene dado por requisito explicito que debe cumplirse por instrucciones del proyecto, esta rige toda la lógica interna, cumpliendo estrictamente con la restricción académica de excluir cualquier ciclo iterativo convencional. Cada algoritmo desarrollado bajo este paradigma se apoya en una estructura rigurosa compuesta por un caso base y un paso recursivo, lo que asegura que la pila de llamadas de Python se gestione de forma controlada y predecible. Este enfoque permite que el sistema descomponga tareas complejas, como la gestión de listas o el renderizado de objetos, en subproblemas mínimos que se resuelven de manera sucesiva hasta alcanzar la condición de parada, garantizando la integridad de la ejecución sin depender de estructuras como for o while.

En el ámbito de la interfaz gráfica desarrollada con Tkinter, la recursividad es vital para mantener el dinamismo de las pantallas sin bloquear el hilo de eventos del usuario. Un ejemplo crítico es la limpieza de los contenedores visuales antes de cada cambio de pantalla, donde la lógica procesa una lista de componentes hijos destruyendo un elemento a la vez e invocándose nuevamente con el remanente de la lista hasta vaciarla por completo. De igual forma, el renderizado de cuadrículas en la selección de

personajes y en el mapa de progreso utiliza un avance recursivo que calcula coordenadas matemáticas y posiciona los nodos visuales secuencialmente, permitiendo que la interfaz se construya de forma coherente a partir de los diccionarios de datos almacenados en el motor del juego.

Finalmente, tanto el procesamiento de información externa como las mecánicas de batalla operan bajo este mismo principio de recursión lineal para asegurar la consistencia del estado global. La carga de activos desde el archivo personajes.csv se realiza mediante una función que consume el iterador del archivo fila por fila, deteniéndose únicamente al agotar los registros, mientras que la construcción de equipos transforma listas de nombres en objetos estadísticos mediante llamadas sucesivas. Asimismo, las funciones de búsqueda y la remoción de personajes tras un knockout en combate se gestionan mediante comparaciones y filtrados recursivos que evitan el uso de métodos iterativos internos de Python.

Gestión de Datos y Memoria del Juego

La gestión de la información en este proyecto se divide entre el almacenamiento de datos estáticos y el control del estado dinámico del juego. Para cumplir con esto, el sistema administra los datos de los 15 personajes de forma externa mediante un archivo en formato .csv, el cual centraliza atributos clave como el nombre, los puntos de vida, la defensa y el ataque. Al guardar esta información fuera del código fuente, el programa puede leer de forma recursiva la franquicia de origen y el rol de combate de cada entidad (ya sea que se desempeñe como atacante, tanque o soporte) facilitando que el motor de datos integre la información al ambiente de combate sin necesidad de modificar la programación principal cada vez que se requiera un ajuste de balance.

En cuanto al desarrollo de la partida, el software se encarga de monitorear el avance del jugador a través un mapa compuesto por cinco tierras distintas (Hollows), donde cada ubicación sirve como punto de referencia para el progreso global. Este control de estado se vincula directamente con el sistema de puntaje, el cual registra cada victoria incrementando el contador del usuario en una unidad por cada personaje del que logra adueñarse tras realizar un knockout exitoso. Al centralizar estas variables, el juego mantiene la consistencia en el flujo de la partida, asegurando que tanto el puntaje acumulado como la posición del jugador se actualicen correctamente conforme se derrotan a los diversos "Hollows" que resguardan cada fragmento de la historia.

Mecánicas de Combate y Generación Aleatoria

El corazón de las batallas es el sistema de combate, que utiliza una fórmula matemática para decidir cuánta vida pierde cada personaje en cada encuentro. El daño se calcula restando la defensa del que recibe el golpe a los puntos de ataque del que golpea, siguiendo la ecuación: $\text{Daño} = \text{Ataque} - \text{Defensa}$. Si por alguna razón el resultado es menor a uno, el código le asigna automáticamente un valor de 1 para asegurar que siempre haya progreso en la pelea y evitar que un combate se quede estancado indefinidamente.

Para que el juego no sea predecible y cumpla con las reglas del proyecto, se utilizó el módulo random de Python para gestionar todos los eventos azarosos del combate. Esto es especialmente importante para la inteligencia del Hollow, ya que, por requerimiento, todas sus decisiones se eligen de

forma aleatoria, asegurando que sus ataques y movimientos no sigan un patrón fijo. Además de la conducta del enemigo, este módulo controla el sistema de golpes críticos: cada vez que alguien ataca, el sistema genera internamente un número al azar; si este cae dentro del 20% de probabilidad de éxito, el ataque se convierte en crítico y el daño total se duplica. Esto añade un factor de sorpresa al juego, ya que un golpe crítico puede cambiar el rumbo de la batalla incluso si el enemigo parece tener la ventaja.

En cuanto a la lógica de turnos, el juego funciona como un intercambio de acciones ordenado y secuencial. El flujo comienza cuando el jugador selecciona su ataque; una vez procesado el daño, el sistema le cede automáticamente el turno al "Hollow". Este ciclo de ataque y respuesta se repite de forma constante hasta que uno de los dos personajes llega a cero puntos de vida y se declara un ganador por knockout.

Control de Interfaz y Eventos

El funcionamiento de la interfaz gráfica se basa en el bucle de eventos de Tkinter, el cual se encarga de gestionar todas las interacciones del usuario, como clics de botones o entradas de texto. En este modelo, el programa no se ejecuta de forma lineal, sino que espera constantemente a que ocurra un evento para reaccionar mediante funciones específicas llamadas callbacks. Esto permite que la aplicación sea interactiva sin necesidad de estructuras complejas de control. Adicionalmente, el uso del método after() permite introducir pequeños retrasos en la ejecución sin detener el programa completamente. Esta técnica es especialmente importante en el sistema de combate, donde se necesita simular turnos entre el jugador y el enemigo sin bloquear la interfaz. Gracias a esto, el juego mantiene una experiencia fluida y responde correctamente a las acciones del usuario.

Finalmente, la arquitectura visual del proyecto se apoya en el uso del objeto Canvas como componente principal, lo que facilita el posicionamiento absoluto de los gráficos y la superposición de capas de texto para una experiencia más inmersiva. El control de la interactividad se gestiona mediante el uso de funciones lambda en los disparadores de los botones, técnica esencial para transmitir argumentos específicos en tiempo de ejecución sin comprometer el flujo lógico del programa. Por último, para optimizar el rendimiento y la persistencia de los recursos, se implementó una estrategia de gestión de memoria visual a través de un caché de referencias; esto previene que el recolector de basura (Garbage Collector) de Python elimine prematuramente los objetos PhotoImage, asegurando la correcta visualización de todos los activos multimedia durante la ejecución del juego.

Análisis de Resultados

El desarrollo del sistema permitió comprobar la funcionalidad de utilizar la recursividad como único método para el control de flujo en una aplicación con interfaz gráfica compleja. Mediante las pruebas de ejecución, se determinó que la modularización del código entre la lógica y la interfaz facilita una respuesta inmediata a los eventos del usuario, manteniendo la integridad del estado global del juego sin experimentar fugas de memoria o cierres inesperados durante la carga dinámica de recursos desde el archivo .csv.

Un aspecto crítico analizado fue el rendimiento de las funciones recursivas en la gestión de la GUI. Se observó que el uso de la recursión para la limpieza y renderizado de widgets en el objeto Canvas opera de manera eficiente dentro de los límites de la pila de llamadas de Python, dado que el volumen de elementos procesados simultáneamente no supera el umbral de desbordamiento. Asimismo, la implementación de funciones lambda y el manejo de un caché de imágenes resultaron ser soluciones técnicas efectivas para evitar la recolección de basura prematura, garantizando la persistencia visual de los activos durante toda la sesión.

Finalmente, el motor de combate demostró una aplicación correcta de las reglas lógicas y matemáticas establecidas. Las pruebas sobre la fórmula de daño y la probabilidad de ataques críticos confirmaron que el uso del módulo random genera una variabilidad de escenarios suficiente para cumplir con el requisito de comportamiento aleatorio del enemigo. En conclusión, los resultados obtenidos reflejan un software robusto que sustituye exitosamente el uso de estructuras iterativas por un enfoque funcional, logrando un sistema escalable y técnicamente alineado con los objetivos académicos del curso.

Bibliotecas Usadas

El desarrollo de este proyecto se apoyó exclusivamente en bibliotecas estándar de Python, evitando la instalación de dependencias externas para garantizar la portabilidad y el bajo acoplamiento del sistema. La biblioteca principal implementada fue Tkinter, la cual proporcionó la infraestructura necesaria para la construcción de la interfaz gráfica de usuario. A través de componentes específicos como el objeto Canvas, se logró un renderizado absoluto y estructurado de los elementos visuales, permitiendo la superposición de imágenes y la gestión de eventos del teclado y ratón.

Complementariamente, se integró el módulo random para introducir la variabilidad matemática requerida en el motor de juego. Esta biblioteca es la encargada de procesar las decisiones aleatorias del sistema, gestionando tanto la selección impredecible de los ataques por parte del Hollow como el cálculo de probabilidades subyacente a la mecánica de golpes críticos. Finalmente, para la carga y persistencia de las estadísticas de los combatientes, se aprovechó el manejo nativo de archivos de texto estructurados mediante funciones de apertura y lectura secuencial, permitiendo extraer la información del archivo CSV y convertirla en diccionarios de datos manipulables.

El proyecto incorpora un sistema de audio independiente ubicado en el módulo /Logic/audio.py, encargado de reproducir la música del juego en momentos clave como el menú principal y las batallas. A diferencia de otras implementaciones más complejas, este sistema utiliza el módulo estándar subprocess de Python para ejecutar un reproductor externo del sistema operativo (aplay), lo que permite reproducir archivos de sonido sin necesidad de instalar librerías adicionales. Esta decisión mantiene la simplicidad del proyecto y cumple con la restricción de utilizar únicamente herramientas nativas del lenguaje.

Además, el sistema implementa un control básico del estado de reproducción mediante variables globales que almacenan el proceso activo y la pista actual. Esto permite evitar que la misma canción se reinicie innecesariamente o que se reproduzcan múltiples audios al mismo tiempo. También se incluye una función para detener la música de forma segura, asegurando que los recursos del sistema se liberen

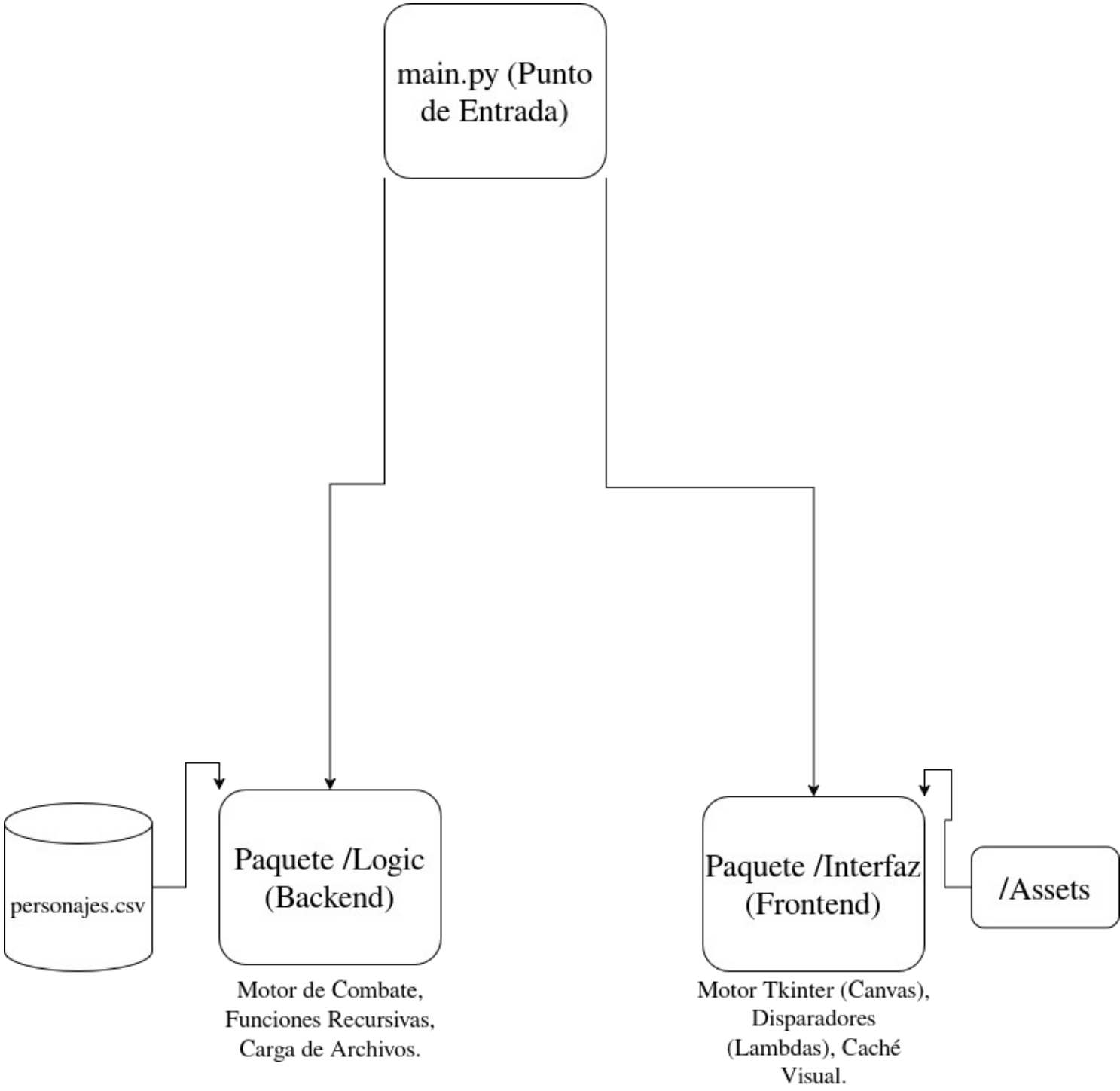
correctamente. **Como limitación, este enfoque depende del sistema operativo, por lo que su funcionamiento está pensado principalmente para entornos Linux.**

Código Asistido y Generado por Inteligencia Artificial

Durante el proceso de desarrollo, se utilizó asistencia de Inteligencia Artificial de forma controlada para resolver obstáculos técnicos avanzados y específicos de la biblioteca Tkinter, los cuales superan el alcance introductorio de la lógica de programación convencional. El primer bloque de código generado por IA corresponde a la gestión de referencias visuales para evadir la recolección de basura (Garbage Collection) nativa de Python. En Tkinter, cuando una imagen se asigna a un componente visual dentro del ámbito de una función local, el lenguaje libera ese espacio de memoria al terminar la ejecución, provocando que las imágenes no se rendericen en pantalla. La solución integrada mediante IA consistió en generar una estructura de caché que adjunta la referencia del archivo multimedia directamente al objeto del widget, garantizando que los activos gráficos de los personajes y escenarios persistan de manera correcta mientras la interfaz se mantenga activa.

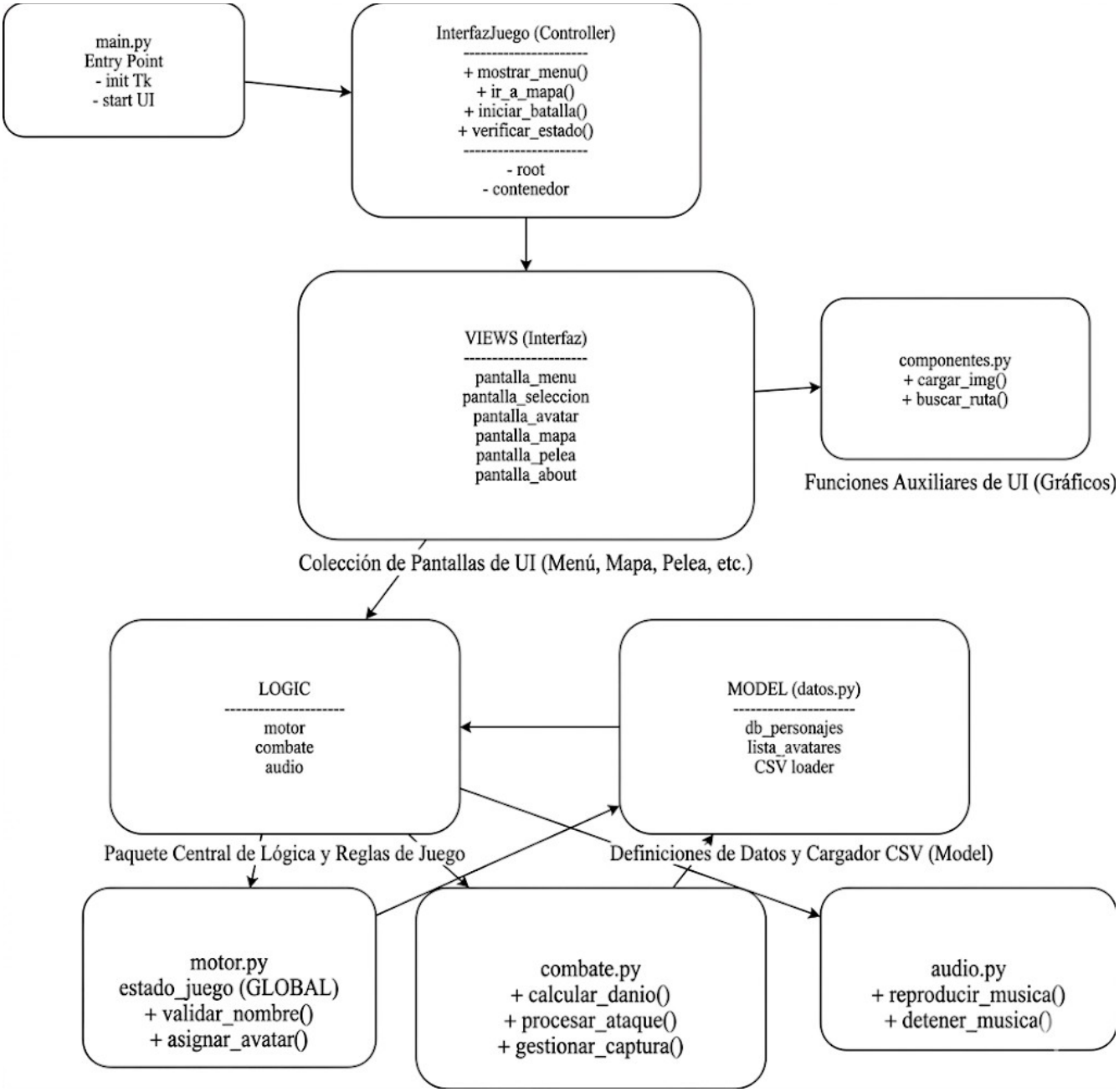
El segundo aporte técnico de la IA se centró en la implementación de funciones anónimas complejas, conocidas como expresiones lambda, para el manejo de eventos dinámicos. Durante la construcción de menús y grillas de forma recursiva, asignar una función con argumentos directamente al disparador de un botón provoca que dicha función se ejecute de forma automática e inmediata durante la carga visual, rompiendo el flujo del juego. Para solucionar este inconveniente, la inteligencia artificial estructuró cierres léxicos (closures) utilizando lambdas. Esta técnica encapsula el llamado a la función lógica, forzando a que parámetros dinámicos, como el nombre de un personaje o las coordenadas de un ataque, se evalúen y envíen al motor de juego única y exclusivamente en el momento en que el usuario interactúa físicamente con el elemento, manteniendo intacta la estabilidad de la recursión.

Diagrama de Arquitectura Básico



Sobre el diagrama de clases

La arquitectura de este proyecto no se rige por un paradigma estrictamente Orientado a Objetos (POO), ya que el uso de clases se limitó de forma exclusiva a la creación y gestión de la interfaz gráfica. Dado que el resto del sistema opera mediante funciones y estructuras de datos, no resulta adecuado incluir un Diagrama de Clases UML convencional. En su lugar, se presenta un Diagrama de Arquitectura Modular, el cual ilustra la organización de los archivos fuente y el flujo de comunicación entre la interfaz visual, la lógica del juego y el modelo de datos.



Literatura y fuentes consultadas

Python Software Foundation. (2024). random — Generate pseudo-random numbers. Documentación oficial de Python 3. Recuperado de https://docs.python.org/3/library/random.html

Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Documentación oficial de Python 3. Recuperado de https://docs.python.org/3/library/tkinter.html

Sergio A. Castaño Giraldo . (2021, 21 de junio). Diccionarios en Python Tutorial [Ejercicios] # 020. YouTube. https://www.youtube.com/watch?v=niGaBxyJCSs